Unité 10 — Cahier de laboratoire (étudiant)

Livrable C — Taint, risque & CVSS · UE « Software Security »

Code UE 7532 14 U32 D1 · Document étudiant · 11 octobre 2026

Cadre. Travail d-analyse ; pas d-exploitation nouvelle.

Lab 10.1 — Tracer la teinte source → sink EXERCICE GUIDÉ

1. Objectifs

- Identifier sources, sinks et assainisseurs dans l'application web (U7/U8).

2. Prérequis

- Cours U10 §1 ; Unités 7/8.

3. Environnement

Image (E) ; bibliothèque `cvss` ; codes des unités précédentes.

4-5. Démarche / commandes

DÉMARCHE

```
# analyse manuelle d'app.py : pour chaque route, reperer
#   SOURCE (request.args[...]) -> propagation -> SINK (execute / check_output / open / format HTML)
```

6. Manipulations

a. Pour chaque route, notez la source, le sink, et s'il existe un assainisseur.

b. Comparez `app.py` et `app_fixed.py` : où l'assainisseur apparaît-il ?

7. Résultats attendus

Tableau source/sink/assainisseur pour les 4 routes ; dans la version corrigée, un assainisseur (paramétrage, validation, confinement, encodage) s'intercale.

8. Explications

Une injection = chemin teinté source→sink sans assainisseur ; corriger = insérer le bon assainisseur au bon endroit.

9. Questions

1. Quelles sources et sinks par route ?
2. Quel assainisseur pour chacune ?
3. Pourquoi l'assainisseur doit-il correspondre au contexte du sink ?

12. Durée

1 h.

10. Corrigé : livrable D, Lab 10.1.

Lab 10.2 — Scorer en CVSS et classer par CWE EXERCICE GUIDÉ

1. Objectifs

- Attribuer un vecteur CVSS v3.1 et un CWE à chaque vulnérabilité des labos.

2. Prérequis

- Lab 10.1 ; Unités 2–8.

3. Environnement

Image (E) ; bibliothèque `cvss` ; codes des unités précédentes.

4-5. Démarche / commandes

SHELL

```
python3 - <<'PY'
from cvss import CVSS3
print(CVSS3("CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H").base_score)  # RCE
PY
```

6. Manipulations

a. Pour chaque vulnérabilité (SQLi, RCE, traversée, XSS…), construisez le vecteur CVSS et calculez le score.

b. Associez le CWE correspondant.

c. Classez par gravité décroissante.

7. Résultats attendus

SQLi 9.1, RCE par débordement de pile 9.8, traversée 7.5, XSS 6.1 (scores de référence) ; CWE : 89, 121, 22, 79 ; injection de commandes Flask : CWE-78.

8. Explications

Le vecteur traduit l'exposition et l'impact ; le score permet la priorisation.

9. Questions

1. Pourquoi le RCE prime-t-il sur le XSS ?
2. Quel rôle jouent UI et S (portée) dans le score du XSS ?
3. CWE vs CVE : différence ?

12. Durée

1 h.

10. Corrigé : livrable D, Lab 10.2.

Lab 10.3 — Registre de risques EXERCICE AUTONOME

1. Objectifs

- Consolider un registre de risques pour l'application auditée.

2. Prérequis

- Labs 10.1–10.2.

3. Environnement

Image (E) ; bibliothèque `cvss` ; codes des unités précédentes.

4-5. Démarche / commandes

DÉMARCHE

```
# tableau : description | CWE | CVSS | exploitabilite | actif | traitement | echeance
```

6. Manipulations

> a. Dressez le registre (une ligne par vulnérabilité).

> b. Proposez un traitement (corriger/atténuer/accepter) et une priorité.

7. Résultats attendus

Registre complet, priorisé, avec traitements justifiés.

8. Explications

Le registre transforme des findings techniques en décisions de gestion du risque.

9. Questions

1. Quelle vulnérabilité traiter en premier, et pourquoi ?
2. Quand « accepter » un risque est-il défendable ?
3. Comment suivre le traitement dans le temps ?

12. Durée

1 h 30.

10. Corrigé : livrable D, Lab 10.3.

Taint issue de meltypot ; CVSS/CWE auteur ; scores calculés (bibliothèque cvss).